Hardware Processing Engines on FPGA: A Survey of Open-Source RISC-V Platforms

Krutarth Patel EE23B137, IIT Madras
Supervisors: Dr. G. Srinivasan, Dr. K. Sankaranarayanan, Prof. N. Chandrachoodan
ee23b137@smail.iitm.ac.in

***Abstract*—Custom, low-cost hardware for AI inference is a major research challenge driven by growing industry demand for edge deployment. Numerous open-source platforms exist for this purpose. This report documents a three-month survey (February–May 2026) of candidate platforms for running a Hardware Processing Engine (HWPE) on an FPGA, conducted as part of the UGRC project at IIT Madras. We evaluated the Google Coral NPU, the PULP Platform (PULPissimo), and several PULP-derived alternatives, documenting the specific technical barriers that prevented deployment in each case. We conclude with a working system built around X-HEEP extended with the Quadrilatero systolic-array co-processor, which achieves simulation-verified matrix multiply and convolution workloads. We identify and fix an RTL handshake deadlock in Quadrilatero that prevented any result from being produced, and provide a fully reproducible Docker-based build environment [1].**

***Index Terms*—RISC-V, hardware accelerator, systolic array, FPGA, HWPE, CV-X-IF, X-HEEP, Quadrilatero, edge AI.**

I. Introduction

The past decade has seen widespread adoption of machine learning across almost every domain. A recurring challenge in deploying these models is satisfying the power and area constraints imposed by the target environment. Embedded and edge devices in particular demand compute units that are both efficient and small. Numerous hardware platforms and software toolchains have been developed to address this, but the landscape is fragmented and difficult to navigate.

From the perspective of a researcher trying to build and deploy an application, the number of options is overwhelming. The central question this study addresses is:

Can a modern, open-source RISC-V SoC be extended with a hardware accelerator, programmed using a software toolchain, and demonstrated on an FPGA?

We constrain ourselves to open-source software and hardware to avoid a dependency on institutional licences or proprietary tools. We further require that the hardware platform support widely-available educational FPGAs. These are arguably the minimum requirements for reproducible research.

We evaluate four candidate platforms: Google Coral NPU, PULPissimo, a cohort of PULP-derived alternatives, and X-HEEP with the Quadrilatero systolic-array co-processor. For each platform we document the specific issues encountered and whether they were resolved. Sections III through V cover the rejected platforms. Section VI covers X-HEEP+Quadrilatero in detail. Section VII documents the RTL bug found and fixed in Quadrilatero. Section VIII presents the experimental results, and Section IX concludes.

II. Platform Selection Criteria

Table I lists the major axes of the design space across the platforms surveyed for this project. The list is not exhaustive.

TABLE I
EDGE-AI HARDWARE DESIGN SPACE.

Axis	Representative options
CPU core	CV32E40PX, Snitch, CVA6, Ibex
Accelerator	Quadrilatero, Gemmini, ITA, RedMulE
Interface	CV-X-IF, HWPE-Stream, AXI
FPGA target	Nexys-A7, Genesys2, ZCU102, PYNQ-Z2
Compiler	GCC, LLVM/Clang, TVM, Deeploy
Simulator	Verilator, QuestaSim, GVSoC

Given the constraints described in Section I, three criteria were chosen for evaluating a given platform:

1) **FPGA support.** The platform must be synthesisable on the Nexys-A7-100t or Nexys-A7-200t without proprietary EDA licences.
2) **Open-source toolchain.** Compilation, simulation, and programming must be achievable with freely available tools.
3) **Accelerator integration.** The SoC must provide a well-documented mechanism for attaching a custom HWPE.

III. Platform 1: Google Coral NPU

A. Background

Google Coral is an edge-AI hardware ecosystem centred on a purpose-built Neural Processing Unit (NPU). It exposes a Python API for running TensorFlow Lite models and is designed for rapid prototyping of neural network inference on embedded hardware.

B. Evaluation and Issues

Coral was evaluated in February 2026 as a potential baseline. The evaluation was brief: none of the three selection criteria could be satisfied.

- **No FPGA flow.** Coral is a proprietary ASIC. There is no RTL netlist and no synthesis flow. The NPU cannot be replicated or studied on an FPGA.
- **Closed architecture.** The NPU's microarchitecture is not documented. Extending it or understanding its performance characteristics requires reverse engineering.
- **No custom-ISA support.** Workloads must reach the NPU through the TFLite-to-Coral compiler. There is no

mechanism for adding custom instructions or attaching a custom accelerator.

C. Verdict

Against the three criteria from Section II:

1) **FPGA support.** Not satisfied. Coral is a proprietary ASIC with no RTL or synthesis flow.
2) **Open-source toolchain.** Partially satisfied for standard models via TFLite, but the NPU internals are closed and undocumented.
3) **Accelerator integration.** Not satisfied. The platform does not support custom instructions or external accelerators.

Coral was abandoned. It fails all three criteria and there is no path to making it work within the constraints of this project.

IV. Platform 2: PULPissimo

A. Background

PULPissimo is a microcontroller SoC from ETH Zurich's PULP (Parallel Ultra Low Power) Platform [2]. It features a RISC-V core and is designed to interface with HWPEs through the standardised HWPE-Stream interface [3]. It has been widely used in embedded AI research and is the most established open-source platform for HWPE work. The platform nominally supports several FPGA boards including the Genesys2 and Nexys families.

B. Evaluation and Issues

Setup was attempted on a modern Ubuntu 22.04 host in early March 2026. When that failed, progressively older environments were tried: Ubuntu 20.04 via a community Docker image (`patata0717/pulpissimo`). Within the Docker container, the PULP SDK compiled and RISC-V code could be emulated using GVSoC, the PULP virtual-platform emulator. RTL simulation could not be achieved.

1) SDK End-of-Life: The main branch of PULPissimo carries no active software SDK. The `pulp-sdk` requires `gcc-5`, which is packaged only for Ubuntu 18. Attempting to build on Ubuntu 20 or 22 fails with incompatible toolchain errors. The last tagged release (June 2021) reduces but does not eliminate dependency conflicts.

2) RTL Simulation Requires QuestaSim: The `common_cells` library used throughout PULPissimo's RTL employs SystemVerilog testbench features (randomisation, coverage, assertions):

```
** Error: ../ips/common_cells/src/id_queue.sv(278):
Questa has encountered an unexpected internal error.
```

ModelSim's trial version does not support these features; QuestaSim (the commercial edition, requiring a licence) is required. Verilator is not officially supported by PULPissimo.

C. Verdict

Against the three criteria from Section II:

1) **FPGA support.** Nominally present in older documentation, but the supported boards (Genesys2, ZCU102) were not available in the lab, and the synthesis flow was never verified with the current codebase.
2) **Open-source toolchain.** Not satisfied. RTL simulation requires QuestaSim, a commercial tool requiring a paid licence. Verilator is not officially supported. The software SDK requires `gcc-5` on Ubuntu 18, making it unusable.
3) **Accelerator integration.** Possible in principle via HWPE-Stream, but unreachable in practice without a working simulation environment.

PULPissimo was abandoned. The hardware design is sound, but failing criteria (2) alone is sufficient to make the platform impractical for reproducible research.

V. Platform 3: PULP-Derived Alternatives

Following the decision to target platforms compatible with the Deeploy compiler framework [4], a broader survey of PULP-derived alternatives was conducted in late March 2026. Table II summarises the findings.

TABLE II
PULP-DERIVED PLATFORM SURVEY. FPGA COL.: FIRST-CLASS SUPPORT; SIM COL.: VERILATOR OR EQUIVALENT OPEN SIMULATOR.

Platform	FPGA	Sim	Blocker
Chimera	–	–	No FPGA flow; docs insufficient.
Siracusa	–	–	No public repository.
Mempool + ITA	–	✓	256 cores; FPGA infeasible.
Snitch Cluster	–	✓	No out-of-the-box FPGA flow.
GAP9	N/A	N/A	Proprietary; not open-source.
Cheshire	✓	✓	No Deeploy backend.

1) Chimera: Chimera is a template architecture for multi-HWPE pipelines and has been used in transformer-accelerator research. However, it lacks documentation for the parts relevant to this project, and no FPGA synthesis flow exists.

2) Mempool and ITA: Mempool is a 256-core RISC-V mesh processor. At full scale, FPGA synthesis is infeasible. A scaled-down variant (`minpool`, 16 cores) supports Verilator but has no FPGA flow.

3) Snitch Cluster: Snitch provides a clean Docker image and Verilator support but is designed primarily for simulation-first and ASIC workflows. Porting to the available FPGA boards would require substantial effort.

4) Cheshire: Cheshire is a Linux-capable RISC-V SoC with FPGA support on several boards and serves as the substrate on which Chimera is built. It came closest to satisfying the criteria but lacks a Deeploy compiler backend, which would be required for the software-stack goal of the project.

A. Summary

Every platform surveyed here lacked either a ready-made FPGA synthesis flow, a working accelerator integration path, or both. X-HEEP, found at the end of March 2026, addresses both gaps.

VI. Platform 4: X-HEEP + Quadrilatero

A. System Overview

X-HEEP (eXtensible Heterogeneous Energy-Efficient Platform) is a configurable RISC-V microcontroller developed at EPFL's ESL lab [5]. It is built around the CV32E40PX core (a 32-bit, four-stage in-order processor) and is purpose-designed for extensibility: a co-processor can be attached through the **CV-X-IF** interface [6] without modifying any MCU RTL. X-HEEP ships with first-class FPGA support for the Nexys-A7, Genesys2, ZCU102, and PYNQ-Z2 boards, and an active Docker-based build system.

Quadrilatero [7] is a systolic-array co-processor implementing the custom `xtheadmatrix` RISC-V ISA extension for matrix operations. The 4×4 systolic array is a weight-static design inspired by the Register-Aware Systolic Array (RASA) architecture [8]. It connects to X-HEEP through CV-X-IF, enabling the CPU to dispatch matrix instructions directly to the accelerator. Quadrilatero includes its own Load-Store Unit (LSU) with direct memory access, making it semi-autonomous once an instruction is accepted.

B. What X-HEEP Does Right

1) FPGA-First Design: Unlike the platforms surveyed earlier, X-HEEP treats FPGA deployment as a primary target rather than an afterthought. Synthesis scripts are maintained for multiple boards and updated with each release. The bitstream generation command is a single `make` invocation:

```
make vivado-fpga FPGA_BOARD=nexys-a7-200t
```

2) CV-X-IF: A Clean Accelerator Interface: The CV-X-IF interface [6] decouples the CPU pipeline from the co-processor completely. The CPU offloads custom instructions without stalling the pipeline (assuming the co-processor responds promptly with an `accept` signal), and the co-processor writes results back through a separate result channel. This means Quadrilatero can be integrated into X-HEEP without touching any MCU RTL, which is a significant advantage over tightly-coupled HWPE-Stream designs where the accelerator must be wired into the SoC fabric directly.

3) Active Docker Toolchain: X-HEEP provides an official `x-heep-toolchain` Docker image that packages Verilator, FuseSoC, and all MCU dependencies. In contrast to PULPissimo, the simulation flow uses Verilator, which is freely available and actively maintained. The image is regularly updated and tested against the current X-HEEP codebase.

C. Environment Setup

A custom `Dockerfile` was written on top of `x-heep-toolchain` to add the libraries required to run Vivado inside the container. Vivado itself is *mounted* as an external volume at runtime rather than bundled into the image, keeping the image size manageable and allowing different Vivado versions to be used without rebuilding.

a) Vivado Version Issue: The X-HEEP FPGA TCL scripts use the `-freq_hz` flag in `create_bd_port`, which was introduced after Vivado 2018.3. The initially available installation was version 2018.3:

```
Unknown option '-freq_hz', please type
'create_bd_port -help' for usage info.
while executing "source xilinx_generate_clk_wizard.tcl"
```

b) Resolution: A Vivado 2022.1 installation was mounted into the container. Once the correct version was in place, MCU generation and bitstream synthesis completed successfully.

D. Verilator Simulation

The X-HEEP simulation model is compiled using FuseSoC and Verilator. The first attempt to compile the model for the X-HEEP+Quadrilatero configuration failed with:

```
%Error: Exiting due to 1 warning(s) [COMBDLY]
```

Verilator 5.x promotes a non-blocking assignment inside a combinational block to an error by default. The offending line is in the CV32E40PX behavioural clock-gate model:

```
if (clk_i == 1'b0) clk_en <= en_i | scan_cg_en_i;
```

a) Resolution: A `/* verilator lint_off COMBDLY */` guard was applied around the offending line in the vendored copy of the behavioural model. The simulation then compiled and ran cleanly.

E. Custom LLVM Compiler Toolchain

The `xtheadmatrix` ISA extension used by Quadrilatero is not present in upstream LLVM or GCC. A patched LLVM must be built from the `xheep_matrix_spec` repository [9]. The build process first compiles `riscv-gnu-toolchain` and then builds LLVM with the custom RISC-V backend. The toolchain is installed into a user-owned host directory that is mounted into the container, so it persists across container runs.

Applications targeting Quadrilatero are compiled with:

```
make app PROJECT=example_matmul_quadrilatero \
  ARCH=rv32imc_zicsr_xtheadmatrix0p1 \
  COMPILER_FLAGS=-menable-experimental-extensions \
  COMPILER=clang CLANG_LINKER_USE_LD=1
```

F. Verification

A matrix multiplication application was compiled and simulated under Verilator. GTKWave was used to inspect the waveforms. The CV-X-IF transaction sequence was verified:

1) The CPU correctly dispatches every `xtheadmatrix` instruction over CV-X-IF. The offload request channel shows valid opcodes and the expected register addresses.
2) Quadrilatero accepts each instruction (`accept = 1` on the issue channel).
3) **The result-valid signal is never asserted.**

The third observation is not an interface error but an internal failure, and is the subject of Section VII.

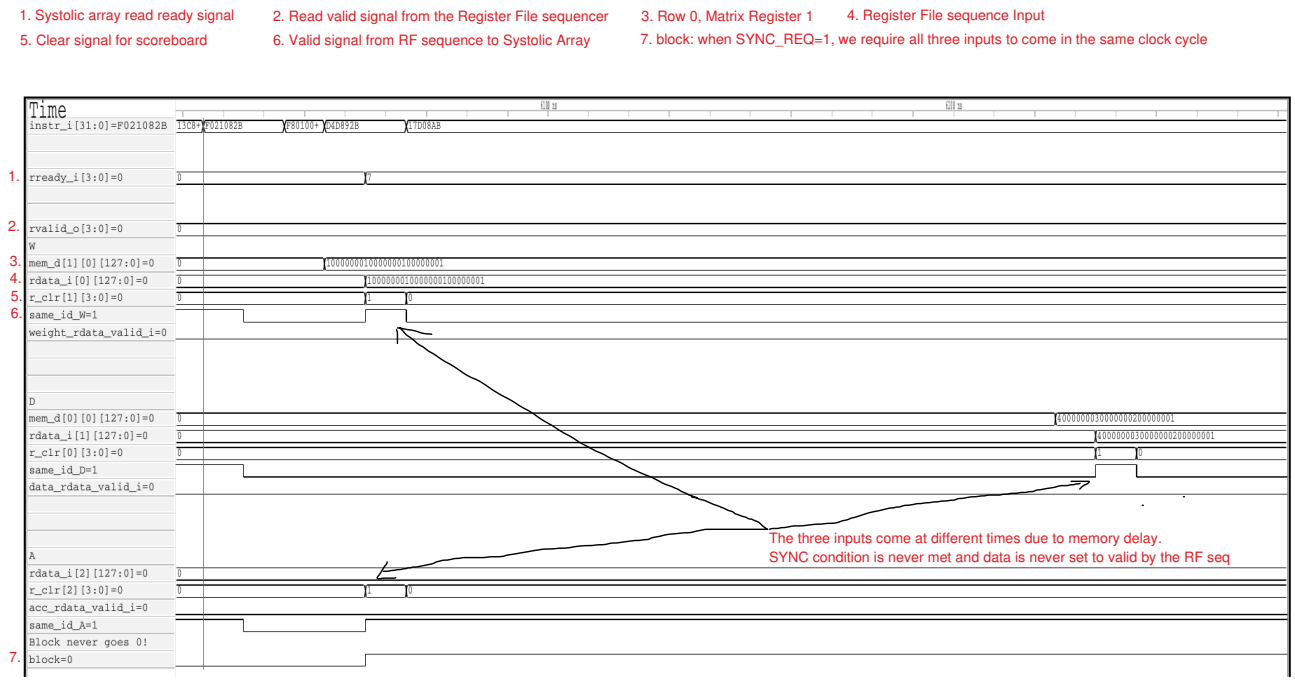

Fig. 1. Waveform with `SYNC_REQ=1`: the RF sequencer blocks forever while waiting for a synchronous data fetch

VII. The Quadrilatero Handshake Bug

A. Default Behaviour

The Register File Sequencer (RF sequencer) is an interface between the systolic array and the register file. It validates accesses to the register file by the systolic array using a scoreboard mechanism.

```
1  if(SYNC_REQ) begin: sa_sync_req
2    logic block      ;
3    logic same_id_acc;
4    logic same_id_A  ;
5    logic same_id_D  ;
6    logic same_id_W  ;
7    same_id_A = rd_id_i[quadrilatero_pkg::SYSTOLIC_ARRAY_A]
8              == scoreboard_q[raddr_i[quadrilatero_pkg::
                   SYSTOLIC_ARRAY_A]]
9                           [rrowaddr_i[quadrilatero_pkg::
                                SYSTOLIC_ARRAY_A]].id;
10   same_id_D = rd_id_i[quadrilatero_pkg::SYSTOLIC_ARRAY_D]
11             == scoreboard_q[raddr_i[quadrilatero_pkg::
                   SYSTOLIC_ARRAY_D]]
12                          [rrowaddr_i[quadrilatero_pkg::
                                SYSTOLIC_ARRAY_D]].id;
13   same_id_W = rd_id_i[quadrilatero_pkg::SYSTOLIC_ARRAY_W]
14             == scoreboard_q[raddr_i[quadrilatero_pkg::
                   SYSTOLIC_ARRAY_W]]
15                          [rrowaddr_i[quadrilatero_pkg::
                                SYSTOLIC_ARRAY_W]].id;
16   if(
17       (rready_i[quadrilatero_pkg::SYSTOLIC_ARRAY_A] && !
              same_id_A) ||
18       (rready_i[quadrilatero_pkg::SYSTOLIC_ARRAY_D] && !
              same_id_D) ||
19       (rready_i[quadrilatero_pkg::SYSTOLIC_ARRAY_W] && !
              same_id_W)
20     ) begin
21     block = 1'b1;
22   end else begin
23     block = 1'b0;
24   end
```

Listing 1. RF sequencer blocking logic with `SYNC_REQ=1`.

By default, `SYNC_REQ` = 1. From Lines 16–21 in Listing 1, the RF sequencer blocks until all three data lines are fetched synchronously. Waveform 1 shows that the `D` data line takes longer to fetch (due to sequential memory access), so `block` is permanently set to 1 and the pipeline stalls indefinitely.

B. Disabling Synchronized Requests

With `SYNC_REQ=0`, waveform 2 shows that the individual data lines become valid at different times. Each line is only valid for one clock cycle before it is popped from the scoreboard queue. This happens because the systolic array holds `rready_i` high permanently, so the handshake completes as soon as data arrives on any given line, regardless of the other lines.

```
1  always_comb begin: rf_block
2    // Weight Read Register Port
3    weight_rdata_ready_o = ff_active_q &~ mask_req  ;
4    ...
5    // Data Read Register Port
6    data_rdata_ready_o   = ff_active_q  &~ mask_req ;
7    ...
8    // Accumulator Read Register Port
9    acc_rdata_ready_o    = ff_active_q &~ mask_req  ;
10   ...
11 end
```

Listing 2. RF output logic (`SYNC_REQ=0`) in systolic array.

From Listing 2 Line 3, `weight_rdata_ready_o` is set whenever the Feed First(FF) stage is active.

```
1  always_comb begin: ctrl_block
2    valid = weight_rdata_valid_i & data_rdata_valid_i &
           acc_rdata_valid_i;
3    clear = ~ff_active_q & ~fs_active_q & ~dr_active_q;
4    ...
```

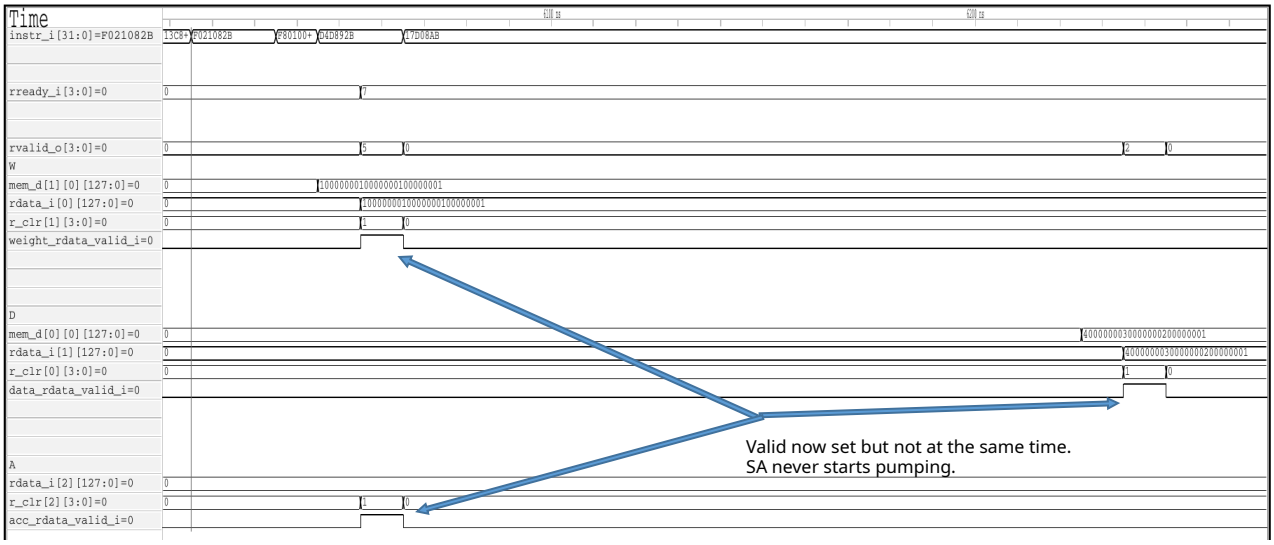

Fig. 2. Waveform with `SYNC_REQ=0`: the systolic array always asserts `ready`, causing data to be popped out of the scoreboard as soon as it is available.

```
5   ff_enable = ff_active_q &  valid                ;
6   pump      = ff_enable | fs_enable | dr_enable
                                                  ;
7   mask_req = (dr_counter_q==$clog2(MESH_WIDTH)'(MESH_WIDTH
        -1))
8           & finished_q & ~finished_ack_i
                                                  ;
9 end
```

Listing 3. Control logic of the systolic array.

However, the control logic requires all three data lines to be valid simultaneously (Listing 3, Line 2). Since they never arrive at the same cycle, `valid` is never asserted and the systolic array never begins pumping (Listing 3, Line 6).

C. Fixing Ready-Valid Handshake

The fix is to change the control logic so that `rready_i` is only asserted once all three data lines are valid, while each line's valid is raised independently as soon as that line's data is available. This is the standard ready-valid handshake: the producer drives valid when its data is ready; the consumer drives ready when it can accept all inputs.

```
1  always_comb begin: rf_block
2    // Weight Read Register Port
3    weight_rdata_ready_o = ff_enable &~ mask_req  ;
4    ...
5    // Data Read Register Port
6    data_rdata_ready_o   = ff_enable  &~ mask_req ;
7    ...
8    // Accumulator Read Register Port
9    acc_rdata_ready_o    = ff_enable &~ mask_req  ;
10   ...
11 end
```

Listing 4. Fixed RF output logic (`SYNC_REQ=0`) in systolic array.

VIII. Results

The measured speedup over the scalar baseline is lower than the $\sim 5\times$ reported in the Quadrilatero paper [7]. The likely cause is that the Weight-Load-Select (WLS) optimisation, which overlaps weight loading with computation, was disabled during debugging to rule out a source of correctness issues. Re-enabling WLS should close the gap.

TABLE III
Verified simulation results. Cycle counts are approximate. Speedup is relative to the scalar baseline.

Matrix size	Scalar (cycles)	Accel. (cycles)	Speedup
4×4	4902	902	5.43
8×8	12802	1502	8.52
16×16	61802	4602	13.43
32×32	433702	30202	14.36

IX. Conclusion

This report surveyed four candidate platforms for running a Hardware Processing Engine on an FPGA using an open-source toolchain. Google Coral was rejected immediately for lacking an FPGA flow. PULPissimo was rejected because its SDK is end-of-life and its simulation flow mandates QuestaSim. Six PULP-derived alternatives were surveyed; all lacked either FPGA support or a compatible compiler backend.

X-HEEP extended with Quadrilatero satisfies all three selection criteria: it has first-class FPGA support, a fully open-source Docker-based toolchain, and a well-specified CV-X-

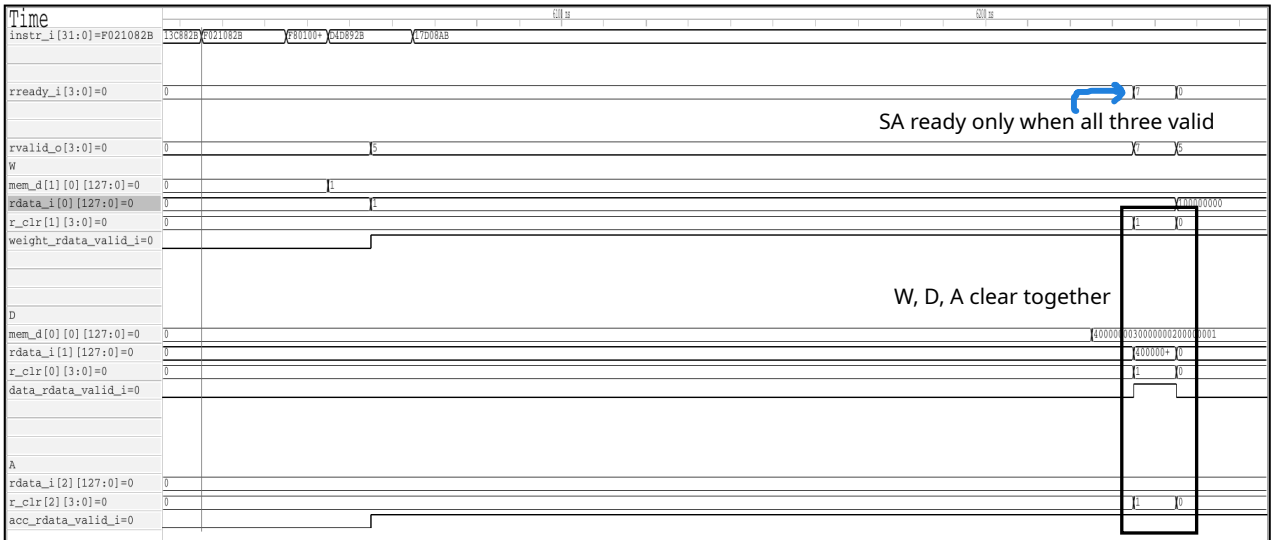

Fig. 3. Waveform of a working simulation. `Valid` set independently by RF sequencer. Systolic array sets `Ready` when `Valid` is set.

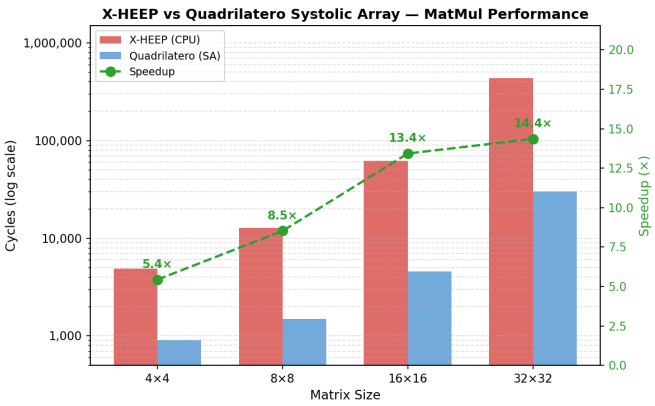

Fig. 4. Cycle counts for scalar (`example_matmul`) vs. accelerated (`example_matmul_quadrilatero`) matrix multiply across input sizes.

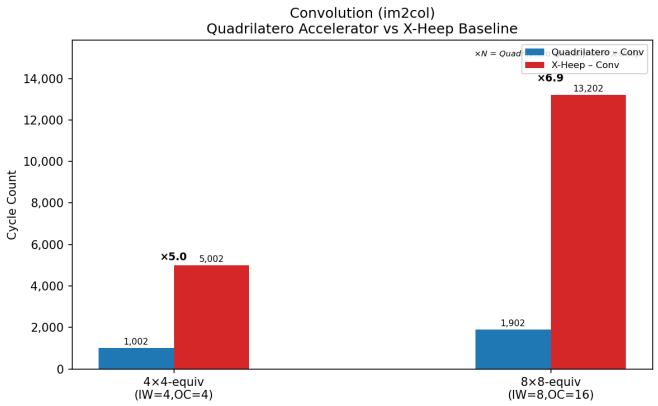

Fig. 5. Cycle counts for convolution (im2col) on Quadrilatero vs. scalar X-HEEP for two small input configurations.

IF accelerator interface that allows the co-processor to be integrated without modifying MCU RTL.

The main takeaway from this survey is that software ecosystem maturity matters as much as hardware quality. PULPissimo's hardware is well-designed; the problem is that the software around it has gone stale. X-HEEP's Docker image, Verilator flow, and Makefile build system made it possible to go from a blank machine to a verified simulation. The full build environment and patched sources are available at [1].

Open problems include FPGA hardware validation against the simulation and integration of the Deeploy compiler framework to enable deployment of full neural network layers from high-level representations.

References

[1] K. Patel, "QuadxHeep: X-HEEP extended with quadrilatero," https://github.com/kuku929/QuadxHeep/tree/main, 2026, accessed: 2026-05-21.

[2] P. D. Schiavone, F. Conti, D. Rossi, M. Gautschi, A. Pullini, E. Flamand, and L. Benini, "Slow and steady wins the race? a comparison of ultra-low-power RISC-V cores for IoT applications," in *IEEE International Workshop on Power and Timing Modeling, Optimization and Simulation (PATMOS)*, 2017, pp. 1–8.

[3] F. Conti, P. D. Schiavone, and L. Benini, "XNOR neural engine: A hardware accelerator IP for 21.6-fj/op binary neural network inference," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 37, no. 11, pp. 2940–2951, 2018.

[4] M. Scherer, P. Rutishauser, G. Colangelo, L. Cavigelli, and L. Benini, "Deeploy: Enabling energy-efficient deployment of small language models and transformers on heterogeneous microcontrollers," *arXiv preprint arXiv:2408.04413*, 2024.

[5] S. Machetti, P. Beltrão, T. Burchi, F. Montagna, G. Ansaloni, D. Atienza, and A. Rahimi, "X-HEEP: An open-source, configurable, and extensible RISC-V microcontroller for the exploration of ultra-low-power edge applications," in *IEEE International Symposium on Circuits and Systems (ISCAS)*, 2023, pp. 1–5.

[6] OpenHW Group, "CV-X-IF: CORE-V extension interface specification," OpenHW Group, Tech. Rep., 2023. [Online]. Available: https://docs.openhwgroup.org/projects/cv-x-if/

[7] M. Cavalcante, L. Ramona, L. Benini, and F. Conti, "Quadrilatero: A high-performance matrix engine for RISC-V processors," in *IEEE International Symposium on Circuits and Systems (ISCAS)*, 2021.

[8] Y. Ding, Z. Huang, J. Li, Y. Ma, D. Niu, and H. Zheng, "RASA: Efficient register-aware systolic array matrix engine for CPU," 2021.

[9] ESL-EPFL, "xheep_matrix_spec: Custom LLVM toolchain for the `xtheadmatrix` ISA extension," https://github.com/esl-epfl/xheep_matrix_spec, 2023.

[10] M. Cavalcante, F. Schuiki, F. Zaruba, M. Schaffner, and L. Benini, "Ara: A 1-GHz+ scalable and energy-efficient RISC-V vector processor with multi-precision floating-point support in 22-nm FD-SOI," *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, vol. 28, no. 2, pp. 530–543, 2020.